

O texto define a arquitetura de software sob duas perspectivas complementares. A primeira foca-se no projeto de alto nível, tratando da organização de componentes de grande porte (módulos, camadas ou serviços) em vez de classes individuais. A segunda perspectiva, inspirada por Ralph Johnson, define a arquitetura como o conjunto de decisões de projeto mais importantes, que são difíceis de reverter uma vez tomadas, como a escolha da linguagem de programação ou do sistema de gestão de base de dados.

O debate histórico entre Andrew Tanenbaum e Linus Torvalds ilustra como estas decisões arquiteturais (monolítica vs. microkernel) têm impacto na evolução de sistemas ao longo de décadas, revelando que escolhas iniciais podem levar a sistemas "inchados" se não houver um controlo rigoroso da estrutura.

A Arquitetura em Camadas é apresentada como um padrão hierárquico onde uma camada apenas utiliza serviços daquela que lhe é imediatamente inferior. Esta separação disciplina as dependências e facilita a manutenção, a substituição de componentes e o reuso (como exemplificado nos protocolos de rede TCP/IP).

Um caso específico discutido é a Arquitetura em Três Camadas, padrão comum em sistemas corporativos distribuídos:

Interface com o Utilizador : Responsável por toda a interação e exibição de dados ao utilizador final.

Lógica de Negócio : Onde são implementadas as regras e validações do domínio do sistema.

Base de Dados: Camada responsável pela persistência e armazenamento das informações.

Originalmente criado para interfaces gráficas em Smalltalk-80, o MVC separa o sistema em três grupos distintos:

Modelo : Classes que gerem os dados e a lógica de domínio, sem dependência da interface.

Visão : Classes responsáveis pela apresentação visual.

Controladora : Intermediação que interpreta os eventos do utilizador e coordena as alterações no Modelo ou na Visão.

A resenha destaca que o MVC favorece a testabilidade e a especialização do trabalho (front-end vs. back-end). O texto clarifica ainda a diferença entre o MVC Clássico e o MVC Web, onde frameworks modernos (como Spring ou Django) adaptam estes conceitos para o ambiente de browsers e servidores.

O texto menciona as SPAs (como o Gmail) como uma evolução moderna do MVC no front-end. Utilizando frameworks como o Vue.js, estas aplicações carregam todo o código inicialmente e utilizam comunicação assíncrona para serem mais interativas, mantendo uma sincronia automática entre modelo e visão através do recurso de two-way data binding.

A transição do Monólito para os Microserviços é apresentada como uma solução para o "gargalo" em empresas que adotam métodos ágeis. Enquanto num monólito o sistema corre como um processo único e partilha memória, nos microserviços o sistema é decomposto em processos independentes que comunicam via interfaces públicas (APIs).

**Evolução Independente:** As equipas podem lançar novas versões de serviços específicos sem necessidade de atualizar ou testar todo o sistema.

**Escalabilidade Granular:** Permite replicar apenas os componentes que apresentam problemas de performance (ex: apenas o serviço de autenticação).

**Persistência Poliglota:** Cada serviço pode utilizar a tecnologia mais adequada (ex: Java com SQL para transações e Python com NoSQL para recomendações).

**Tolerância a Falhas:** Uma falha num serviço específico (como o de recomendações) não impede o funcionamento das funções principais do sistema (falha parcial).

O capítulo associa os microserviços à Lei de Conway, sugerindo que a arquitetura do software acaba por espelhar a estrutura organizacional da empresa: equipas pequenas, autónomas e descentralizadas.